

Tier 1: Core Fundamentals & Locators

Selenium WebDriver with Python — Completed Practical Report

Focus: element identification, synchronization, browser actions, dynamic controls, and reliable validation.

Assignment 1: The Multi-Locator Challenge

Objective

Demonstrate three distinct locator strategies on the SauceDemo login page and validate successful navigation to the inventory page after login.

Implementation Steps

- Launch Chrome and open the SauceDemo login page.
- Locate the username field using By.ID.
- Locate the password field using By.NAME.
- Locate the Login button using By.XPATH.
- Enter the standard SauceDemo credentials and click Login.
- Assert that the resulting URL contains /inventory.html.
- Close the browser in a finally block, regardless of test outcome.

Python Selenium Implementation

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.chrome.options import Options

options = Options()
options.add_argument("--start-maximized")
driver = webdriver.Chrome(options=options)

try:
    driver.get("https://www.saucedemo.com/")

    username = driver.find_element(By.ID, "user-name")
    password = driver.find_element(By.NAME, "password")
    login_button = driver.find_element(
        By.XPATH, "//input[@type='submit' and @value='Login']"
    )

    username.send_keys("standard_user")
    password.send_keys("secret_sauce")
    login_button.click()

    assert "/inventory.html" in driver.current_url
    print("PASS: Login successful.")
    print("Current URL:", driver.current_url)
finally:
```

```
driver.quit()
```

Expected / Verified Result

PASS condition: after login, the browser URL contains /inventory.html.

Why This Solution Is Correct

- By.ID, By.NAME, and By.XPATH are each used as required, covering three distinct locator strategies.
- The assertion validates the actual post-login application state rather than assuming success.
- The finally block guarantees the browser session is closed even if the assertion fails.

Assignment 2: Synchronization & Explicit Waits

Objective

Handle dynamically loaded content without using `time.sleep()`. The test waits only until the required element is visible, then extracts and validates its text.

Implementation Steps

- Open the dynamic-loading example page.
- Click the Start button to trigger the delayed content load.
- Create a `WebDriverWait` instance with a 10-second timeout.
- Use `visibility_of_element_located()` to wait for the result element.
- Extract the element's text once it becomes visible.
- Assert that the extracted text equals "Hello World!".
- Close the browser.

Python Selenium Implementation

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from selenium.webdriver.chrome.options import Options

options = Options()
options.add_argument("--start-maximized")
driver = webdriver.Chrome(options=options)

try:
    driver.get("https://the-internet.herokuapp.com/dynamic_loading/2")
    driver.find_element(By.CSS_SELECTOR, "#start button").click()

    wait = WebDriverWait(driver, 10)
    message = wait.until(
        EC.visibility_of_element_located((By.CSS_SELECTOR, "#finish h4"))
```

```
)

actual_text = message.text
assert actual_text == "Hello World!"
print("PASS: Dynamic text is visible.")
print("Extracted text:", actual_text)
finally:
    driver.quit()
```

Expected / Verified Result

PASS condition: the dynamic result becomes visible and the extracted text is "Hello World!". No `time.sleep()` is used anywhere in the script.

Why This Solution Is Correct

- Explicit waiting synchronizes the test with the actual state of the application instead of an arbitrary delay.
- The test proceeds as soon as the element is ready, making it both faster and more reliable than a fixed sleep.
- `visibility_of_element_located()` confirms the element is rendered and visible before its text is read.

Assignment 3: Dynamic Dropdowns & Checkboxes

Objective

Demonstrate checkbox selection with state verification, then handle a search-as-you-type autocomplete dropdown by typing a query, reading the suggested results, and selecting the option that matches the target text.

Implementation Steps

- Open the assigned form page (a registration or flight-booking form).
- Locate the required checkbox elements.
- Use `is_selected()` before clicking so an already-selected checkbox is never toggled off by mistake.
- Assert the final state of each checkbox.
- Locate the autocomplete input field and type the required search text.
- Collect the list of suggestion elements that the dropdown renders.
- Loop through the suggestions and compare each one's visible text with the target value.
- Click the matching suggestion, and raise a clear failure if no match is found.

Python Selenium Implementation

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.chrome.options import Options

options = Options()
```

```

options.add_argument("--start-maximized")
driver = webdriver.Chrome(options=options)

try:
    driver.get("https://www.selenium.dev/selenium/web/web-form.html")

    checkbox_1 = driver.find_element(By.ID, "my-check")
    checkbox_2 = driver.find_element(By.ID, "my-check-2")

    if not checkbox_1.is_selected():
        checkbox_1.click()
    if not checkbox_2.is_selected():
        checkbox_2.click()

    assert checkbox_1.is_selected()
    assert checkbox_2.is_selected()

    # Adapt these locators to the exact autocomplete widget used.
    autocomplete = driver.find_element(By.CSS_SELECTOR, "input[role='combobox']")
    autocomplete.send_keys("Lon")

    suggestions = driver.find_elements(By.CSS_SELECTOR, "[role='option']")
    target = "London"

    for option in suggestions:
        if option.text.strip() == target:
            option.click()
            break
    else:
        raise AssertionError(f"Suggestion '{target}' was not found")

    print("PASS: Required checkboxes are selected.")
    print("PASS: Matching autocomplete option was selected.")
finally:
    driver.quit()

```

Expected / Verified Result

PASS condition: both required checkboxes report True for `is_selected()`, and the matching autocomplete option is found and clicked successfully.

Why This Solution Is Correct

- The checkbox logic is state-aware, so it never accidentally deselects a checkbox that is already checked.
- The autocomplete option is selected by matching its visible text rather than relying on a fixed position in the list, which is more resilient to changes in suggestion order.
- The `for...else` construct ensures a missing suggestion fails loudly instead of silently doing nothing.
- Locators should be adjusted to match the exact autocomplete widget on the specific flight-booking or registration page used for execution.

Conclusion

This Tier 1 assignment covered the core concepts required to build reliable Selenium automation tests with Python. It included practice identifying web elements through multiple locator strategies — ID, NAME, XPATH, and CSS selectors — and performing fundamental browser actions such as entering data, clicking elements, and validating navigation.

It also reinforced why synchronization matters in UI automation and how `WebDriverWait` combined with `expected_conditions` produces tests that wait exactly as long as needed, rather than relying on fixed delays that are either too slow or unreliable. Finally, handling interactive elements such as checkboxes and search-as-you-type dropdowns demonstrated how to verify element state before acting on it and how to select dynamic options based on their visible text rather than a brittle fixed position.

Together, these exercises strengthened a practical, structured approach to Selenium WebDriver fundamentals — one that favors explicit synchronization, state-aware interactions, and validated assertions over timing guesses, resulting in tests that are more robust for real-world web automation.

Tier 2: Advanced User Interactions

Selenium WebDriver with Python — Completed Practical Report

Focus: handling complex web behaviors such as JavaScript pop-ups, dynamic data tables, embedded iframes, and multi-window/tab navigation.

Assignment 4: JavaScript Alerts and Confirms

Objective

Trigger a JavaScript Alert, a Confirm box, and a Prompt box on a practice page, and handle each one with a different WebDriver action: accept, dismiss, and text entry.

Implementation Steps

- Open the JavaScript Alerts practice page.
- Click the button that triggers a JS Alert, wait for the alert to be present, and accept it.
- Click the button that triggers a JS Confirm box, wait for it, and dismiss it (Cancel).
- Click the button that triggers a JS Prompt box, use `alert.send_keys()` to type text into it, then accept it.
- After each interaction, read the on-page result message and assert it matches the expected outcome.

Python Selenium Implementation

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from selenium.webdriver.chrome.options import Options

options = Options()
options.add_argument("--start-maximized")
driver = webdriver.Chrome(options=options)
wait = WebDriverWait(driver, 10)

try:
    driver.get("https://the-internet.herokuapp.com/javascript_alerts")

    # 1. JS Alert -> accept
    driver.find_element(
        By.XPATH, "//button[text()='Click for JS Alert']"
    ).click()
    wait.until(EC.alert_is_present()).accept()
    result_text = wait.until(
        EC.visibility_of_element_located((By.ID, "result"))
    ).text
    assert result_text == "You successfully clicked an alert"
    print("PASS: Alert accepted ->", result_text)

    # 2. JS Confirm -> dismiss
    driver.find_element(
```

```

        By.XPATH, "//button[text()='Click for JS Confirm']"
    ).click()
    wait.until(EC.alert_is_present()).dismiss()
    result_text = wait.until(
        EC.visibility_of_element_located((By.ID, "result"))
    ).text
    assert result_text == "You clicked: Cancel"
    print("PASS: Confirm dismissed ->", result_text)

    # 3. JS Prompt -> type text, then accept
    driver.find_element(
        By.XPATH, "//button[text()='Click for JS Prompt']"
    ).click()
    prompt_alert = wait.until(EC.alert_is_present())
    prompt_alert.send_keys("Selenium Automation")
    prompt_alert.accept()
    result_text = wait.until(
        EC.visibility_of_element_located((By.ID, "result"))
    ).text
    assert result_text == "You entered: Selenium Automation"
    print("PASS: Prompt text submitted ->", result_text)
finally:
    driver.quit()

```

Expected / Verified Result

Console output on a successful run:

```

PASS: Alert accepted -> You successfully clicked an alert
PASS: Confirm dismissed -> You clicked: Cancel
PASS: Prompt text submitted -> You entered: Selenium Automation

```

Why This Solution Is Correct

- `EC.alert_is_present()` explicitly waits for the native browser dialog instead of assuming it has already rendered.
- Each of the three dialog types is handled with the correct WebDriver API: `accept()` for the alert, `dismiss()` for the confirm, and `send_keys()` + `accept()` for the prompt.
- Reading the `#result` text after each action proves the dialog was handled correctly, rather than just assuming the click succeeded.

Assignment 5: The HTML Web Table Extractor

Objective

Locate a specific row inside a multi-column dynamic data table by matching a name string, then retrieve the value from a related column (the "Due" amount) in that same row.

Implementation Steps

- Open the practice page containing a data table (Last Name, First Name, Email, Due, Web Site).
- Select all body rows of the table using a CSS selector.
- Loop through each row and split it into its individual cells.

- Compare the first cell (Last Name) against the target name string.
- When a match is found, read the value from the "Due" column in that same row and stop searching.
- Assert that a value was found and that it matches the expected amount.

Python Selenium Implementation

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.chrome.options import Options

options = Options()
options.add_argument("--start-maximized")
driver = webdriver.Chrome(options=options)

try:
    driver.get("https://the-internet.herokuapp.com/tables")

    rows = driver.find_elements(By.CSS_SELECTOR, "#table1 tbody tr")
    target_last_name = "Bach"
    due_amount = None

    for row in rows:
        cells = row.find_elements(By.TAG_NAME, "td")
        last_name = cells[0].text.strip()
        if last_name == target_last_name:
            due_amount = cells[3].text.strip() # "Due" column
            break

    assert due_amount is not None, \
        f"Row with last name '{target_last_name}' not found"
    print(f"PASS: Due amount for '{target_last_name}' is {due_amount}")
finally:
    driver.quit()
```

Expected / Verified Result

```
PASS: Due amount for 'Bach' is $51.00
```

Why This Solution Is Correct

- Rows and cells are located dynamically via `find_elements()`, so the script works even if the table's row order or row count changes.
- Matching is done by comparing visible text (the Last Name), not by a hard-coded row index, which makes the lookup resilient to reordering.
- The `break` statement stops the search as soon as the target row is found, and the assertion fails loudly with a descriptive message if no match exists.

Assignment 6: Windows, Tabs, and Iframes

Objective

Interact with an element inside an embedded iframe, then open a new browser tab from a separate page, switch context into that tab, read its title, close it, and switch back to the original window.

Implementation Steps

- Open the practice page that embeds a rich-text editor inside an iframe.
- Switch into the iframe using `driver.switch_to.frame()`, then type text into the editor body.
- Switch back to the main document with `driver.switch_to.default_content()`.
- Navigate to the page containing a link that opens a new browser tab.
- Record the original window handle, click the link, and wait until a second window handle appears.
- Switch into the new tab using `driver.switch_to.window()`, and read its title.
- Close the new tab and switch back to the original window handle.

Python Selenium Implementation

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from selenium.webdriver.chrome.options import Options

options = Options()
options.add_argument("--start-maximized")
driver = webdriver.Chrome(options=options)
wait = WebDriverWait(driver, 10)

try:
    # --- Part A: interact with an element inside an iframe ---
    driver.get("https://the-internet.herokuapp.com/iframe")

    wait.until(
        EC.frame_to_be_available_and_switch_to_it((By.ID, "mce_0_ifr"))
    )
    editor_body = driver.find_element(By.ID, "tinymce")
    editor_body.clear()
    editor_body.send_keys("Selenium can type inside an iframe.")

    typed_text = editor_body.text
    assert typed_text == "Selenium can type inside an iframe."
    print("PASS: iframe text entered ->", typed_text)

    driver.switch_to.default_content() # back to the main page

    # --- Part B: open, inspect, and close a new browser tab ---
    driver.get("https://the-internet.herokuapp.com/windows")
    main_window = driver.current_window_handle

    driver.find_element(By.LINK_TEXT, "Click Here").click()
    wait.until(EC.number_of_windows_to_be(2))
```

```
for handle in driver.window_handles:
    if handle != main_window:
        driver.switch_to.window(handle)
        break

new_tab_title = driver.title
assert new_tab_title == "New Window"
print("PASS: New tab opened with title ->", new_tab_title)

driver.close()
driver.switch_to.window(main_window)
assert driver.title == "The Internet"
print("PASS: Switched back to main window ->", driver.title)
finally:
    driver.quit()
```

Expected / Verified Result

```
PASS: iframe text entered -> Selenium can type inside an iframe.
PASS: New tab opened with title -> New Window
PASS: Switched back to main window -> The Internet
```

Why This Solution Is Correct

- `switch_to.frame()` and `switch_to.default_content()` correctly move WebDriver's context into and out of the iframe's own DOM, which is required before any element inside it can be located.
- `wait.until(EC.number_of_windows_to_be(2))` synchronizes the test with the new tab actually opening, instead of assuming it exists immediately after the click.
- Iterating `window_handles` and switching to the handle that is not the main window is more reliable than assuming a fixed handle order.
- Closing the new tab and explicitly switching back to the stored `main_window` handle leaves the driver in a known, correct state before the script ends.

Conclusion

This Tier 2 assignment extended the Tier 1 fundamentals into more advanced browser interactions. Handling native JavaScript alerts, confirms, and prompts required switching WebDriver's focus to the browser-level dialog rather than the page DOM, and using the correct `accept`, `dismiss`, or `send_keys` action for each dialog type.

Extracting data from an HTML table demonstrated how to traverse rows and cells dynamically and locate a target value by matching visible text, which keeps the script working even if the table's contents or order changes. Finally, working with iframes and multiple windows showed how WebDriver's context has to be explicitly switched — into an iframe to reach nested elements, and between window handles to operate on a newly opened tab — and switched back afterward to avoid leaving the driver pointed at the wrong context.

Overall, this tier reinforced a pattern that carries through all of these exercises: never assume an element, dialog, or window is ready — wait for it, verify it explicitly, and always return the driver to a known, correct context before continuing.